Fundamentals of theory of computation I.

Lecture 2

Krisztián Tichler
ktichler@inf.elte.hu

Grammars – reminder

Definition

A (generative) **grammar** is a quadruple $G = (N, \Sigma, P, S)$, where

- N and Σ are disjoint alphabets (i.e., $N \cap \Sigma = \emptyset$). N is called the alphabet of **nonterminal** symbols, while Σ is the alphabet of **terminal** symbols.
- $S \in N$ is called the **start symbol** (or initial symbol),
- P is a set of derivation rules $x \to y$, where $x \in (N \cup \Sigma)^* N (N \cup \Sigma)^*, y \in (N \cup \Sigma)^*$.

Note, that this is a slightly reformulated definition based on the following.

Observation: $(N \cup \Sigma)^* - \Sigma^* = (N \cup \Sigma)^* N (N \cup \Sigma)^*$.

Grammars – reminder

Definition

Let $G = (N, \Sigma, P, S)$ be a grammar and $u, v \in (N \cup \Sigma)^*$. v can be **derived** form u **in one step** in G, denoted by $u \Rightarrow_G v$, if $u = u_1 x u_2$ and $v = u_1 y u_2$ hold, where $u_1, u_2 \in (N \cup \Sigma)^*$ and $x \to y \in P$.

Let $G = (N, \Sigma, P, S)$ be a grammar and $u, v \in (N \cup \Sigma)^*$. v can be **derived** (in multiple steps) from u if either $u = v$ or there exist an integer $n \geq 1$ and words $w_0, \ldots w_n \in (N \cup \Sigma)^*$, such that $w_{i-1} \Rightarrow_G w_i$ $(1 \leq i \leq n)$, $w_0 = u$ and $w_n = v$. Notation: $u \Rightarrow^*_G v$.

Sentential form: a word derivable from the start symbol.

$\Rightarrow_G$ and $\Rightarrow^*_G$ are often replaced by $\Rightarrow$ and $\Rightarrow^*$, respectively.

Definition

Let $G = (N, \Sigma, P, S)$ be a grammar. Then $L(G) := \{w \mid S \Rightarrow^*_G w, w \in \Sigma^*\}$ is called the **language generated by the grammar** G.

Chomsky's classification of grammars

Let $G = (N, \Sigma, P, S)$ be a grammar. G is a **grammar of type** i ($i = 0, 1, 2, 3$) if the following conditions hold for the set of rules P.

- case $i = 0$: no further restrictions,
- case $i = 1$:
  (1) all rules of P are of the form $u_1 A u_2 \to u_1 v u_2$, where $u_1, u_2, v \in (N \cup \Sigma)^*$, $A \in N$ and $v \neq \varepsilon$ hold,

  (2) there is only one possible exception: P **may** contain the rule $S \to \varepsilon$, but in this case S does not occur on the right hand side of any rule in P.
  We shall call this "**restricted ε rule**" condition (or shortly "**RER**").
- case $i = 2$: all rules of P are of the form $A \to v$, where $A \in N$ and $v \in (N \cup \Sigma)^*$ hold,
- case $i = 3$: all rules of P are of the form $A \to uB$ or of the form $A \to u$, where $A, B \in N$ and $u \in \Sigma^*$ hold.

Let $\mathcal{G}_i$ denote the class of grammars of type i ($i = 0, 1, 2, 3$).

Classification of grammars

Example: Let $G = (N, \Sigma, P, S)$ be a grammar, where $N = \{S, A, B\}, \Sigma = \{a, b\}$ and $P = \{S \to ASB, S \to \varepsilon, AB \to BA, BA \to AB, A \to a, B \to b\}$. We check the type conditions rule by rule.

$S \to ASB$	0, $1^\bullet$, 2
$S \to \varepsilon$	0, $1^\bullet$, 2, 3
$AB \to BA$	0
$BA \to AB$	0
$A \to a$	0, 1, 2, 3
$B \to b$	0, 1, 2, 3

$^\bullet$: Rules 1 and 2 **TOGETHER** violates the RER condition. If these 2 rules appear together, then $G \notin \mathcal{G}_1$.

Rules 3 and 4 excludes all types but 0, therefore $G \in \mathcal{G}_0$ and $G \notin \mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_3$ hold.

Classification of grammars

Let G be a grammar. These are the cases possible.

- $G \in \mathcal{G}_3, \mathcal{G}_2, \mathcal{G}_1, \mathcal{G}_0$.
  Locally, all rules are of type 3, RER condition holds.
- $G \in \mathcal{G}_3, \mathcal{G}_2, \mathcal{G}_0$.
  Locally, all rules are of type 3, RER condition does not hold.
- $G \in \mathcal{G}_2, \mathcal{G}_1, \mathcal{G}_0$.
  Locally, all rules are of type 2 and there is a non-type 3 rule.
  RER condition holds.
- $G \in \mathcal{G}_2, \mathcal{G}_0$.
  Locally, all rules are of type 2 and there is a non-type 3 rule.
  RER condition does not hold.
- $G \in \mathcal{G}_1, \mathcal{G}_0$.
  Locally, all rules are of type 1 and there is a non-type 2 rule.
  RER condition holds.
- $G \in \mathcal{G}_0$.
  There is a non-type 1 rule or all rules are of type 1, there is a non-type 2 rule but RER condition does not hold.

Chomsky's classification of languages

Noam Chomsky (1928 –) is a linguist, philosopher, political activist. The "father of modern linguistics". He created the concept of generative grammars and also classified them.

(Wikipedia)

$\mathcal{L}_i := \{L \mid \exists G \in \mathcal{G}_i,$ such that $L = L(G)\}$ is the language class of type i languages. Its elements are calley **type i languages**. $(i = 0, 1, 2, 3)$.

So, type i languages are those languages that can be generated by a type i grammar. $(i = 0, 1, 2, 3)$

Naming the grammar and language classes

- Type 0 grammars are called **phrase-structure** (or unrestricted) **grammars**, referring to their origin in linguistics.
- Type 1 grammars are called **context-sensitive grammars** as their rules are like a nonterminal A can be replaced by a word v only in the appropriate context u_1 and u_2.
- Type 2 grammars are called **context-free grammars** as their rules are like a nonterminal A can be replaced by a word v regardless of the context.
- Type 3 grammars are called **regular grammars**, referring to their connection to regular expressions.

Language classes of type 0,1,2,3 languages are also called the language class of

- **recursively enumerable**,
- **context-sensitive** (CS),
- **context-free** (CF),
- **regular**

languages, repectively.

CF Grammars in practice

BNF (Backus-Naur Form) **John Backus, Peter Naur ALGOL 60**

BNF is a metasyntax notation for context-free grammars, often used to describe the syntax of languages used in computing, e.g, for defining programming languages.
Elements of rules:

- ⟨name⟩, **concepts** to be defined (or **non-terminals**)
- ::=, used for the separation of left and right hand sides.
- string elements (or **terminals**)

Each rule is of the following form. There is a single concept on the left hand side followed by ::=, followed by one or more finite sequence of terminals and nonterminals, separated by |.

So, BNF corresponds to context-free grammars. Example:

⟨identifier⟩::= ⟨letter⟩ | ⟨letter⟩ ⟨id-end⟩
⟨id-end⟩::= ⟨letter⟩ | ⟨digit⟩ | ⟨letter⟩ ⟨id-end⟩ | ⟨digit⟩ ⟨id-end⟩
⟨letter⟩::=*a* | ··· | *z*
⟨digit⟩::=0 | ··· | 9

Chomsky hierarchy

According to the definitions of grammar types we have
$\mathcal{G}_3 \subseteq \mathcal{G}_2 \subseteq \mathcal{G}_0$ and $\mathcal{G}_1 \subseteq \mathcal{G}_0$.
This implies $\mathcal{L}_3 \subseteq \mathcal{L}_2 \subseteq \mathcal{L}_0$ and $\mathcal{L}_1 \subseteq \mathcal{L}_0$.

The following theorem will be proved later.

Chomsky hierarchy of languages

$$\mathcal{L}_3 \subset \mathcal{L}_2 \subset \mathcal{L}_1 \subset \mathcal{L}_0.$$

We need the following to prove this theorem.

1. $\mathcal{L}_2 \subseteq \mathcal{L}_1$,
2. strict inclusion $\mathcal{L}_3 \subset \mathcal{L}_2$,
3. strict inclusion $\mathcal{L}_2 \subset \mathcal{L}_1$,
4. strict inclusion $\mathcal{L}_1 \subset \mathcal{L}_0$,

1 – 3 to be proved in this course, 4 to be proved in FOTOC2.

(Weakly) equivalent grammars and languages

Each grammar generates a single language. On the other hand, some languages might be generated by several grammars.

Definition

Two grammars G_1 and G_2 are called **equivalent**, if they generate the same language, i.e. $L(G_1) = L(G_2)$ holds.

Definition

Two languages L_1 and L_2 are called **weakly equivalent**, if they differ in at most the word ε, i.e., $(L_1 - L_2) \cup (L_2 - L_1) \subseteq \{\varepsilon\}$.

Definition

Two grammars G_1 and G_2 are called **weakly equivalent**, if they generate weakly equivalent languages, i.e., $(L(G_1) - L(G_2)) \cup (L(G_2) - L(G_1)) \subseteq \{\varepsilon\}$.

Equivalent grammars

Examples

In the following examples we have $N \subseteq \{S, S', A, B\}$, $\Sigma = \{a, b\}$, start symbol S. So, only the sets of rules to be given. Two rules, $\alpha \to \beta$ and $\alpha \to \gamma$ to be written in a more compact, $\alpha \to \beta \,|\, \gamma$ form.

a) $S \to ASB \,|\, \varepsilon,\ AB \to BA,\ BA \to AB,\ A \to a,\ B \to b$
type 0 grammar

b) $S \to ASB \,|\, \varepsilon,\ AB \to BA,\ A \to a,\ B \to b$
type 0 grammar

c) $S \to S' \,|\, \varepsilon,\ S' \to AS'B \,|\, AB,\ AB \to BA,\ A \to a,\ B \to b$
type 0 grammar (RER condition holds)

d) $S \to SS \,|\, aSb \,|\, bSa \,|\, \varepsilon$.
type 2 grammar

$L(G) = L_{\mathrm{EQ}} := \{u \in \{a, b\}^* \,|\, N_a(u) = N_b(u)\}$. (4 times.)

There's no type 3 grammar G with $L(G) = L_{\mathrm{EQ}}$. (Proved later.)

Equivalent grammars

Examples

(a) was checked on Lecture 1, and it is easy to see, that (b) and (c) are equivalent with (a). So, we prove only (d).

The rules of G are the following. $S \to SS \mid aSb \mid bSa \mid \varepsilon$.

$L(G) \subseteq L_{\mathbf{EQ}}$

By induction on the length of the shortest derivation we prove that every sentential form $w \in \{a, b, S\}^*$ has the property $N_a(w) = N_b(w)$.

Let the length of the shortest derivation be n. The case $n = 0$ is trivial. Assume, that the statement holds for all sentential forms having a shortest derivation less than n.

Let w' be the last but one sentential form in a derivation of w of length n. Then the shortest derivation of w' is at most $n - 1$. By induction, we have $N_a(w') = N_b(w')$. Whatever the last rule applied is it keeps the balance of a's and b's.

Equivalent grammars

Examples

The rules of G are the following. $S \to SS \mid aSb \mid bSa \mid \varepsilon$.

$L(G) \supseteq L_{\mathbf{EQ}}$

Let $w \in L_{\mathrm{EQ}}$, where $|w| = n$. By induction on n we prove $w \in L(G)$.

The case $n = 0$ is trivial. Assume, that the statement holds for all words of L_{EQ} shorter than n. We have 3 cases for $n \geq 1$.

Case 1: $w = aw'b$ holds for some $w' \in L_{\mathrm{EQ}}$. By induction we have $S \Rightarrow^* w'$. So, we have $S \Rightarrow aSb \Rightarrow^* aw'b = w$.

Case 2: $w = bw'a$. Similarly to case 1.

Case 3: $w = aw'a$ or $w = bw'b$ holds for some $w' \in L_{\mathrm{EQ}}$. Then there exist $u, v \in \{a, b\}^+$ with $w = uv$ and $u, v \in L_{\mathrm{EQ}}$. This holds by the observation, that prefixes of length 1 and $n - 1$ of w have different signs for the amount of a's minus the amount of b's.

By induction we have $S \Rightarrow^* u$ and $S \Rightarrow^* v$. So,
$S \Rightarrow SS \Rightarrow^* uS \Rightarrow^* uv = w$. holds.

Eliminating terminals from LHS of rules

All grammars can be transformed into a normal form having only nonterminals on the left hand sides of the rules.

Theorem

For every grammar $G = (N, \Sigma, P, S)$ there is an equivalent grammar $G' = (N', \Sigma, P', S)$ of the same type with the property $x \in (N')^+$ for all $x \to y \in P'$.

Proof: Type 3 and type 2 grammars are already of this form.

For $i = 0, 1$ let $\bar{\Sigma} = \{\bar{a} \mid a \in \Sigma\}$. It can be assumed, that $N \cap \bar{\Sigma} = \emptyset$. Let $N' := N \cup \bar{\Sigma}$. Substitute all occurances of $a \in \Sigma$ by $\bar{a}$ in all rules of P. (Even on the right hand sides!). Let P_1 denote the set of modified rules. Let $\bar{P} = \{\bar{a} \to a \mid a \in \Sigma\}$ and $P' := P_1 \cup \bar{P}$. There are no terminals on the right hand sides of the rules of P'.

We claim, that G' is equivalent with G, i.e., $L(G) = L(G')$.

Eliminating terminals from LHS of rules

$L(G) \subseteq L(G')$: If $u = t_1 \cdots t_n \in L(G)$, $t_i \in \Sigma (1 \leq i \leq n)$, than we have $S \Rightarrow^*_{G'} \bar{t_1} \cdots \bar{t_n}$ by applying the corresponding modified rules. Using the appropriate rules of $\bar{P}$ we get $S \Rightarrow^*_{G'} u$, i.e., $u \in L(G')$.

$L(G) \supseteq L(G')$:

Let a homomophism $h : (N' \cup \Sigma)^* \to (N \cup \Sigma)^*$ be defined as follows. Let $h(x) := x$ for all $x \in (N \cup \Sigma)$ and $h(\bar{a}) := a$ for all $\bar{a} \in \bar{\Sigma}$.

Assume, that $u \Rightarrow_{G'} v$ holds by applying a rule of $r \in P'$. There are 2 cases.

- either $r \in \bar{P}$, then $h(u) = h(v)$,
- or $r \in P_1$, then $h(u) \Rightarrow_G h(v)$.

So $u \Rightarrow_{G'} v$ implies $h(u) \Rightarrow^*_G h(v)$. (In 0 or 1 steps, in fact.)

Eliminating terminals from LHS of rules

By an induction, it is easy to see, that

$u \Rightarrow^*_{G'} v$ implies $h(u) \Rightarrow^*_G h(v)$.

Therefore, if $S \Rightarrow^*_{G'} w$, $w \in \Sigma^*$ holds, then
$S = h(S) \Rightarrow^*_G h(w) = w$ holds as well.

In the case of G being of type 1, G' is of type 1, too.

Remark: Although it was not needed for the proof, this type 0 transformation works for type 2 grammars as well, resulting in a type 2 grammar. So, for all grammars of type 0,1 or 2 we can assume that terminals are occuring only in rules of type $A \to a$, where $a \in \Sigma, A \in N$.

Closure properties

Definition

Operations union, concatenation and iterative closure are called **regular operations**.

A family of languages $\mathcal{L}$ is said to be **closed** under the n-ary operation $\varphi : (L_1, \ldots, L_n) \mapsto \varphi(L_1, \ldots, L_n)$ if $\varphi(L_1, \ldots, L_n) \in \mathcal{L}$ holds for every $L_1, \ldots, L_n \in \mathcal{L}$.

Theorem

$\mathcal{L}_i$ is closed under the regular operations ($i = 0, 1, 2, 3$).

Proof: A language being of type i means that there is a grammar of type i generating the language. So, our task is to give 3 times 4 = 12 constructions of grammars

(1) generating the union, concatenation, iterative closure of the generated languages, respectively,

(2) having the same type as the original grammar(s).

Closure properties

According to our previous theorem we can assume that no terminals occur on the left hand side of any rules.

(Note, that we shall use this assumption only in the cases of type 0 concatenation, type 0 and type 1 iterative closures.)

It can be assumed as well, that all pairs of grammars have a disjoint nonterminal alphabet, and they are disjoint from the union of the terminal alphabets.

Union:

Let $L_k \in \mathcal{L}_i$ and let $G_k = (N_k, \Sigma_k, P_k, S_k) \in \mathcal{G}_i$ be grammas with the property $L(G_k) = L_k$ $(k = 1, 2)$.

Cases $i = 0, 2, 3$:

Let $S_0 \notin N_1 \cup N_2$.

$G_\cup := (N_1 \cup N_2 \cup \{S_0\}, \Sigma_1 \cup \Sigma_2, P_1 \cup P_2 \cup \{S_0 \to S_1 \mid S_2\}, S_0)$.

Then $G_\cup$ is of type i and $L(G_\cup) = L_1 \cup L_2$.

Closure properties

Union:

Case $i = 1$:

The only problem for type 1 grammars, is that if one of the grammars have an ε-rule (i.e., the right hand side of the rule is ε), of course with the RER condition, then the start symbol of that grammar becomes a non-start symbol with an ε-rule, which is not allowed.

Let $G_k = (N_k, \Sigma_k, P_k, S_k) \in \mathcal{G}_1$ be grammars satisfying $L(G_k) = L_k - \{\varepsilon\}$ $(k = 1, 2)$.

We can get such grammars, by deleting the rules $S_1 \to \varepsilon$ and $S_2 \to \varepsilon$ rules from P_1 and P_2 type 1 grammars generating L_1 and L_2, respectively, if they are present.

Now, consider the previous construction for $G_\cup$. In the case of $\varepsilon \in L_1$ or $\varepsilon \in L_2$ add $S \to S_0 \,|\, \varepsilon$ to the set of rules and let S be the start symbol.

Closure properties

Concatenation:

Let $L_k \in \mathcal{L}_i$ and $G_k = (N_k, \Sigma_k, P_k, S_k) \in \mathcal{G}_i$ with the property $L(G_k) = L_k$ $(k = 1, 2)$.

Cases $i = 0, 2$:

Let $S_0 \notin N_1 \cup N_2$.

$G_c := (N_1 \cup N_2 \cup \{S_0\}, \Sigma_1 \cup \Sigma_2, P_1 \cup P_2 \cup \{S_0 \to S_1 S_2\}, S_0)$.

It is easy to see, that we have $L(G_1)L(G_2) \subseteq L(G_c)$.

$L(G_1)L(G_2) \supseteq L(G_c)$ holds due to $N_1 \cap N_2 = \emptyset$ and our previous theorem on the left hand sides of the rules.

In fact, one can prove by induction on the length of derivation, that all sentential forms of G_c are of $w_1 w_2$, where $S_k \Rightarrow^*_{G_k} w_k$ $(k = 1, 2)$.

This statement holds after the first step of derivation. Assume, that all sentential forms of G_c are of the above form after n steps of derivation. Since all left hand sides of the rules of $P_1 \cup P_2$ are from $N_1^+ \cup N_2^+$ this will hold after the $(n + 1)$st step, too.

Closure properties

Concatenation:

Case $i = 3$: The problem with the previous construction is that $S_0 \to S_1 S_2$ is not a type 3 rule.

$$P_1^+ := \{A \to uS_2 \,|\, A \to u \in P_1, A \in N_1, u \in \Sigma_1^*\} \cup \{A \to uB \,|\, A \to uB \in P_1, A, B \in N, u \in \Sigma_1^*\},$$

$$G_c := (N_1 \cup N_2, \Sigma_1 \cup \Sigma_2, P_1^+ \cup P_2, S_1).$$

I.e., consider the terminating rules of P_1, the rules containg only terminals on the right hand side. Concatenate these rules with the start symbol of G_2. By applying such a rule we can start a derivation of a word of $L(G_2)$ on the right of a terminal word derivated in G_1.

On the other hand, let vS_2 be the first sentential form in the derivation $u \in L(G_c)$ containing S_2. Then $u = vw$ and due to $N_1 \cap N_2 = \emptyset$ we have $S_1 \Rightarrow_{G_1}^* v$ and $S_2 \Rightarrow_{G_2}^* w$.

So, $L(G_c) = L(G_1)L(G_2)$ holds.

Closure properties

Concatenation:

Case $i = 1$:

We face the same problem as in the case of union. ε-rules, if they exist from S_1 or S_2 are not from the start symbol anymore, so they might become non-type 1 rules in the type 0,2 construction of G_c.

Let $L'_k = L_k - \{\varepsilon\}$ $(k = 1, 2)$ and consider the type 0,2 construction of G_c for L'_1, L'_2. So we have $L'_1 L'_2 \in \mathcal{L}_1$.

Then

$L_1 L_2 =$

	$\varepsilon \notin L_2$	$\varepsilon \in L_2$
$\varepsilon \notin L_1$	$L'_1 L'_2$	$L'_1 L'_2 \cup L'_1$
$\varepsilon \in L_1$	$L'_1 L'_2 \cup L'_2$	$L'_1 L'_2 \cup L'_1 \cup L'_2 \cup \{\varepsilon\}$

$L_1 L_2 \in \mathcal{L}_1$, since we have $L'_1 L'_2,\ L'_1,\ L'_2,\ \{\varepsilon\} \in \mathcal{L}_1$ and according to our proof $\mathcal{L}_1$ is closed under the union operation.

Closure properties

Iterative closure:

Let $L \in \mathcal{L}_i$ and $G = (N, \Sigma, P, S) \in \mathcal{G}_i$ with the property $L(G) = L$.

Case $i = 2$:

Let $S_0 \notin N$ be a new nonterminal.

Then $G_* := (N \cup \{S_0\}, \Sigma, P \cup \{S_0 \to SS_0 \mid \varepsilon\}, S_0)$ generates L^*.

Case $i = 3$:

Since $S_0 \to SS_0$ is not a type 3 rule, we need something else.

$P_* := \{A \to uS \mid A \to u \in P, u \in \Sigma^*\}$,

i.e., consider all terminating rules of G, and add the start symbol S of G to the right.

Let $S_0 \notin N$ be a new nonterminal.

$G_* := (N \cup \{S_0\}, \Sigma, P \cup P_* \cup \{S_0 \to S \mid \varepsilon\}, S_0)$.

So, we can either start generating a new terminal word of $L(G)$ by the rules of P_* to the right, or we can finish the derivation by the original rules.

Closure properties

Iterative closure:

Cases $i = 0, 1$:
Unfortunately, the previous, type 2 construction does not work for type 1 and 0. Due to the possible nonsolitary symbols on the left hand sides there might be rules having an effect through iteration boundaries. This may result in generated words not in the iterative closure.

Case $\varepsilon \notin L$: Let $S_0, S_1 \notin N$.

$G_* := (N \cup \{S_0, S_1\}, \Sigma, P \cup P' \cup P'' \cup P''', S_0)$, ahol

$P' = \{S_0 \to \varepsilon \mid S \mid S_1 S\}$, // 0, 1 or more iterations

$P'' = \{S_1 a \to S_1 S a \mid a \in \Sigma\}$, // middle iterations

$P''' = \{S_1 a \to S a \mid a \in \Sigma\}$. // last iteration

Closure properties

Claim: Sentential forms of G_* are either of the form

(a) $S_1 u_1 \cdots u_n$ $(n \geq 1, S \Rightarrow_G^* u_k, 1 \leq k \leq n)$ or of the form

(b) $u_1 \cdots u_n$ $(n \geq 0, S \Rightarrow_G^* u_k, 1 \leq k \leq n)$,

where the first letter of u_k is in Σ for all $2 \leq k \leq n$.

Observe, that $L(G_*) = L(G)^*$ immediately follows from the claim.

(1) First, we prove by induction on n, that all words of forms (a) or (b) can be derived in G_*. It is easy to check (a) for $n = 1$ and (b) for $n = 0, 1$.

(a) By induction we have $S_0 \Rightarrow^* S_1 u_2 \cdots u_n$. If u_2 starts with a terminal then $S_0 \Rightarrow^* S_1 S u_2 \cdots u_n$ holds. Since $S \Rightarrow_G^* u_1$ we have $S_0 \Rightarrow^* S_1 u_1 u_2 \cdots u_n$.

(b) Let $n \geq 2$. By induction we have $S_0 \Rightarrow^* S_1 u_2 \cdots u_n$. If u_2 starts with a terminal then $S_0 \Rightarrow^* S u_2 \cdots u_n$ holds. Since $S \Rightarrow_G^* u_1$ we have $S_0 \Rightarrow^* u_1 u_2 \cdots u_n$.

Closure properties

(2) Words of form (a) or (b) becomes a word of form (a) or (b) by applying a rule from $P' \cup P'' \cup P'''$.

In the case of applying a rule from P, the left hand side of the applied rule should be a subword of one of u_k's, where $1 \leq k \leq n$. This holds by the fact, that all the left hand sides of the rules of P is in N^+, but all the subwords of the sentential form not being a subword of any u_k's contain at least one terminal.

So the next word in the derivation remains being of the form either (a) or (b).

In the case of type 1, G_* is of type 1 as well.

So we have proved the case $\varepsilon \notin L$.

Case $\varepsilon \in L$:
Observe, that $(L - \{\varepsilon\})^* = L^*$ holds for all languages L.

So, it is enough to construct a grammar G' such that $L(G') = L(G) - \{\varepsilon\}$. (Paying attention to the type of G').

Closure properties

For $i = 1$ just remove the rule $S \to \varepsilon$ if it is present.

For $i = 0$ let $P_\varepsilon = \{p \in P \mid p = \alpha \to \varepsilon, \alpha \in (N \cup \Sigma)^*\}$ be the set of ε-rules of G.

Let $P_1 := (P - P_\varepsilon) \cup P'$, where

$P' = \{uX \to X, Xu \to X \mid X \in (N \cup \Sigma), u \to \varepsilon \in P\}$.

(P' has only CS rules, so if G is a type 1 grammar, then we have the same for G' as well.)

Then $L(G_1) = L(G) - \{\varepsilon\}$, where $G_1 = (N, \Sigma, P_1, S)$.

To see this, one can check the following.

(1) Any application of a rule in P_ε in a derivation of a non-empty word of $L(G)$ can be simulated by the application of one of the corresponding rules (the same rule, but in an environment) in P'.

(2) Any rule of G_* has the same effect as one of the rules of G.